

Lecture: 1000

SRAM vs DRAM & Architectures

**Why Computers Have So Many Kinds of
Memory**

From a 6-transistor cell to the memory wall — the trade-offs that shape every computer.

CSA222: Modern Computer Architecture

4. Von Neumann

One memory holds both instructions and data, reached by a single bus. The stored-program idea that started it all (1945).

The Stored-Program Concept

The radical 1945 idea: keep the program IN memory, as numbers, right alongside the data.

BEFORE vs AFTER

Before: computers were 'wired' for one task — reprogramming meant rewiring by hand.

After (von Neumann): the program is stored in the same memory as data, encoded as numbers.

*To run a different program, you just load different numbers.
Software as we know it was born.*

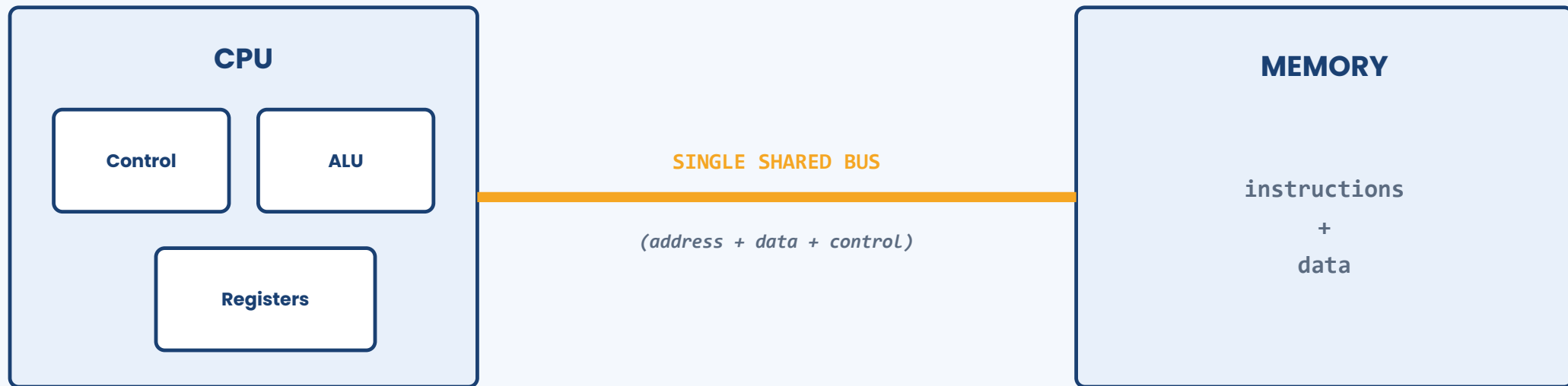
ONE MEMORY, TWO ROLES

0x00	lw r1, 0x40	instr
0x04	add r3, r1, r2	instr
0x08	sw r3, 0x44	instr
0x40	0x0000002A	data
0x44	0x00000000	data

Same memory, same address space — instructions and data live side by side.

Von Neumann Architecture

One memory, one bus — everything shares a single path between CPU and memory.



Instructions and data take turns on the ONE bus — the defining feature, and the built-in limit.

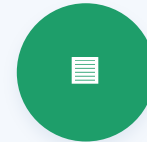
The Key Insight: Programs ARE Data

Because code lives in memory as numbers, it can be treated like any other data.



Compilers & assemblers

A program that writes another program — it outputs instructions as data into memory.



Loading & the OS

The OS loads a program by copying its instruction-bytes into memory, then jumps to them.



Just-in-time compilation

JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.



Self-modifying code

Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.

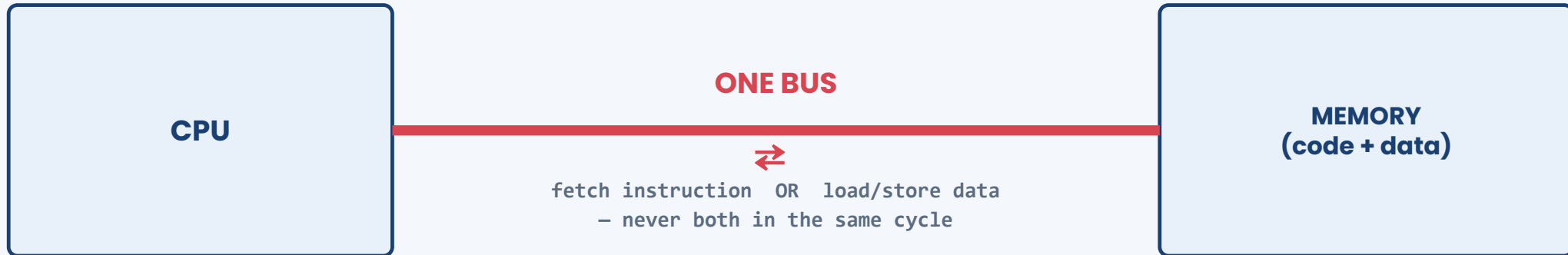
This blurring of code and data is von Neumann's superpower — and the root of whole classes of security exploits.

5. The Von Neumann Bottleneck

One bus, two jobs: instructions and data can't both cross at once. This single limit shapes modern architecture.

The Von Neumann Bottleneck

Every instruction fetch and every data access share ONE bus — so they must take turns.



The CPU can compute far faster than the single bus can feed it — so the bus becomes the limit.

This traffic jam on the memory bus is 'the Von Neumann bottleneck' — the term coined by John Backus in 1977.

Why It Limits Performance

A fast CPU stalls whenever the bus is busy — computation waits on communication.



Bus contention

Instruction fetch and data access compete for the same wires. One must wait for the other.



Idle compute units

While the ALU waits for the bus, it does nothing. Raw compute power sits unused.



Caps achievable IPC

Instructions-per-cycle is limited not by the ALU but by how fast the bus delivers work.



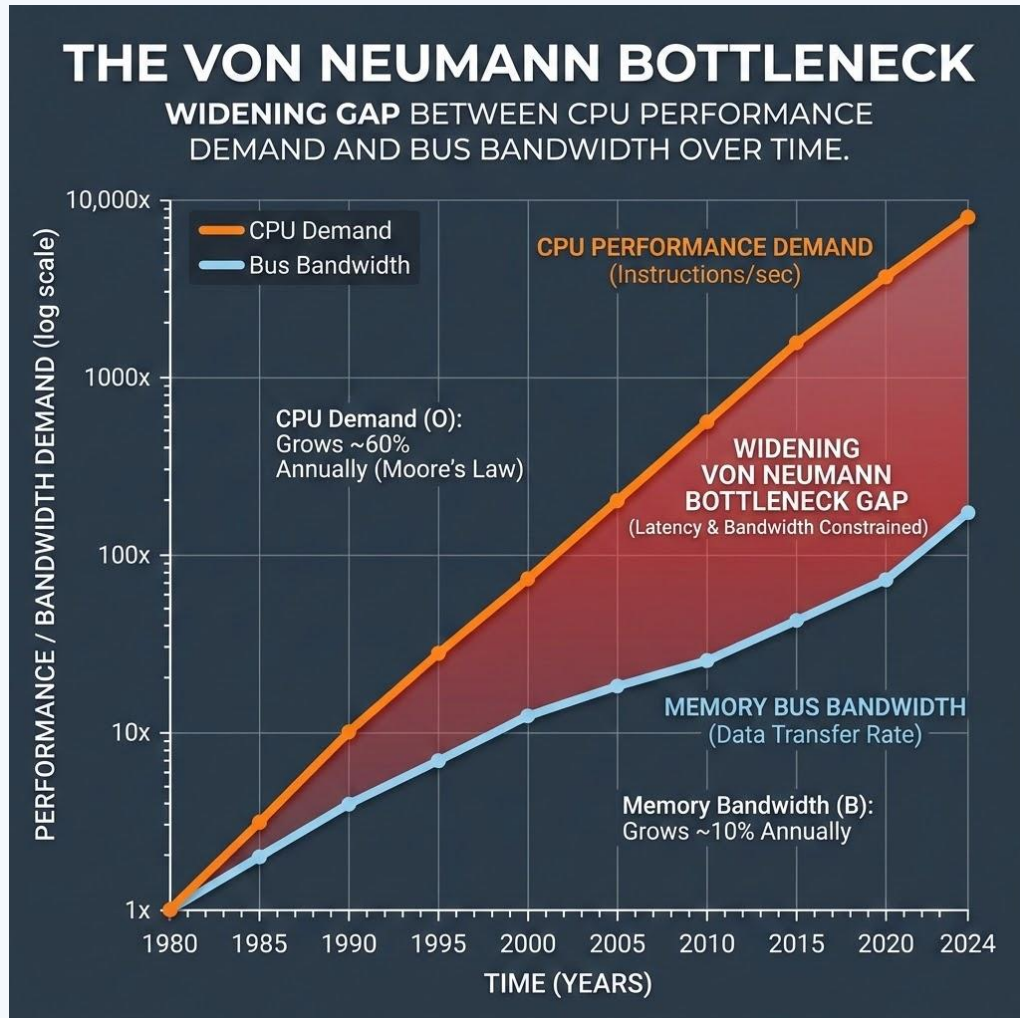
Worsens with CPU speed

Faster CPUs need data faster — but the bus doesn't scale as quickly. The gap grows.

The bottleneck is why caches, wider buses, and Harvard-style splits exist — all attempts to widen this pinch.

Why It Gets Worse Over Time

CPU speed raced ahead of memory bandwidth for decades — the bottleneck tightened.



THE RESPONSE

Architects fought back with:

- Caches — keep hot code & data on-chip (Module 5).
- Wider & faster buses — move more bits per cycle.
- Harvard-style splits — separate paths for code and data.

The rest of today — and much of this course — is about that last idea.

Quick Check

Show of hands — no notes.

Q. The Von Neumann bottleneck exists because...

A

The ALU is too slow

B

One bus is shared by code & data

C

Memory is too small

D

There are too many registers

✓ *they take turns on one path*

The single shared bus means an instruction fetch and a data access can't happen simultaneously — that's the bottleneck.

3. Harvard Architecture

Separate memories and buses for instructions and data — so the CPU can fetch and load at the same time.

Harvard Architecture

Two memories, two buses — instructions and data never compete for the same path.



Instruction fetch (I-bus) and data access (D-bus) happen in the SAME cycle — no contention.

Two Buses, Twice the Throughput

The whole point: parallel access. What took two cycles now takes one.

VON NEUMANN

one bus, take turns

- Cycle 1: fetch instruction.
- Cycle 2: access data.
- Two cycles for fetch + data.
- Bus is the bottleneck.

HARVARD

two buses, in parallel

- Cycle 1: fetch instruction AND access data.
- Both buses active at once.
- One cycle for fetch + data.
- Up to 2× memory throughput.

The cost: two memories and two bus systems — more wires, more pins, more silicon. Nothing is free.

One Cycle, Two Transfers

A timeline makes the advantage obvious — Harvard overlaps what Von Neumann serialises.



Same work, half the time — because the two transfers ride separate buses simultaneously.

Where Harvard Shines

When predictable, high-throughput access matters more than flexibility.



Digital Signal Processors

DSPs stream data through fixed algorithms (filters, FFTs). Parallel code+data fetch keeps the pipeline fed.



Microcontrollers

The ATmega328 (Arduino Uno) is true Harvard — flash for code, SRAM for data, separate buses.



Real-time systems

Deterministic timing matters. No bus contention means predictable, repeatable cycle counts.



Embedded & IoT

Small, fixed programs in flash + small data in RAM map naturally onto the Harvard split.

Von Neumann vs Harvard

The trade-off in one table: simplicity & flexibility vs throughput & predictability.

	VON NEUMANN	HARVARD
Memory	One (unified)	Two (separate I & D)
Buses	One shared	Two independent
Fetch + data	Sequential (2 cycles)	Simultaneous (1 cycle)
Throughput	Bottlenecked	Up to 2× higher
Cost / complexity	Lower	Higher (more wires/pins)
Flexibility	High (code = data)	Lower (separate spaces)
Typical use	General-purpose CPUs	DSPs, microcontrollers

Neither is 'better' — general CPUs pick flexibility, embedded/DSP picks throughput. Modern chips blend both...

6. Modified Harvard

What your laptop actually is: Von Neumann at main memory, Harvard inside the chip. The best of both.

The Real World: Modified Harvard

Modern CPUs are Von Neumann on the outside, Harvard on the inside.

TWO LEVELS, TWO ANSWERS

Outside the chip: one unified DRAM holds code and data — pure Von Neumann. Simple, flexible, cheap.

Inside the chip: the L1 cache is **SPLIT** into a separate instruction cache (L1-I) and data cache (L1-D) — Harvard-style.

So the CPU fetches code and data in parallel from L1, but everything still lives in one memory underneath.

BEST OF BOTH

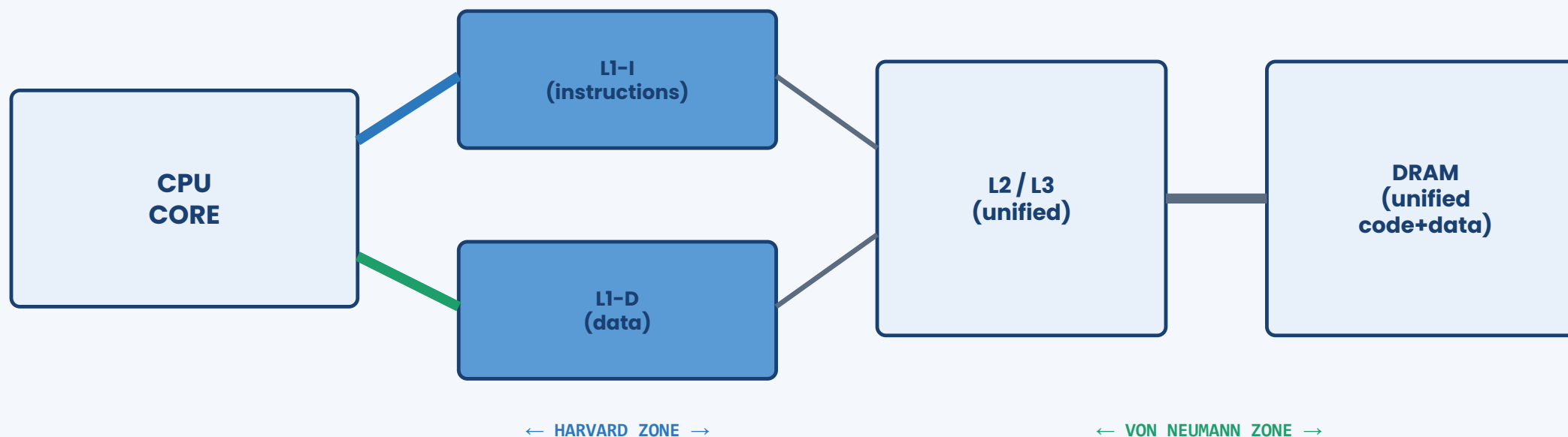
Flexibility of Von Neumann:
one address space, code = data, easy to program & load.

Speed of Harvard:
split L1 caches feed the pipeline code + data every cycle, no contention.

This hybrid is why the split L1 exists — a direct preview of Module 5.

Split L1-I and L1-D Caches

Where the two worlds meet: unified memory below, split caches above.



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

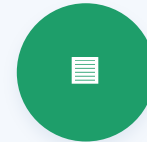
Why This Hybrid Wins

It captures each architecture's strength exactly where that strength matters most.



Parallel L1 access

Split L1-I/L1-D lets the pipeline fetch an instruction and a data word every cycle — Harvard speed where it counts.



Unified main memory

One DRAM address space keeps loading, linking, and programming simple — Von Neumann flexibility.



Cache coherence handled

Hardware keeps the split caches consistent with unified memory, so software sees one memory.



Scales to real workloads

General-purpose speed AND generality — which is why essentially every modern CPU does this.

Key Takeaways

Five sentences — and Module 2 is complete.

- 01** SRAM stores a bit in a 6-transistor latch: fast, non-destructive, no refresh — but bulky and expensive.
- 02** DRAM stores a bit as charge on a capacitor (1T1C): tiny and cheap and dense — but slow, destructive, and needs refresh.
- 03** SRAM builds caches (L1/L2/L3); DRAM builds main memory — each chosen for its trade-off, not because one is 'better'.
- 04** The memory hierarchy layers small-fast over big-slow, and locality lets a small cache catch most accesses.
- 05** The memory wall — CPUs outpacing DRAM for decades — is the problem the whole memory hierarchy exists to fight.

Practice & Homework

Draw the cells, place the caches, and reason about the numbers.

01

Cell Sketch

Draw a 6T SRAM cell and a 1T1C DRAM cell. Label transistors, capacitor, bit lines, word line.

02

Refresh Math

A DRAM row refreshes every 64 ms with 8192 rows. On average, how often must one row be refreshed?

03

Plot It

Plot access time vs capacity (log-log) for registers, L1, L2, L3, DRAM, SSD, HDD.

EXIT TICKET

1. List three reasons SRAM is faster than DRAM.
2. Why must DRAM be refreshed but SRAM need not be?
3. Order these by latency: L2, DRAM, registers, L1, SSD.

Rapid-Fire Recap

Three questions — shout the answers.

Q1

An SRAM cell uses how many transistors?

6

Q2

DRAM needs periodic...

refresh

Q3

Caches are built from...

SRAM

All three instant? You've completed Module 2 — Module 3 turns to the instruction set: the CPU's vocabulary.

END OF L08 • MODULE 2 COMPLETE

Fast, big, cheap — pick two.

NEXT — MODULE 3, L09

Computer Organization & the Instruction Set — how a program becomes machine instructions, and the vocabulary a CPU actually understands.